Geometric Computer Vision

Earl Wong

References

- Multi View Geometry in Computer Vision, Hartley and Zisserman
- Lecture Notes, Shree Nayer

Problem

- Recover 3D scene structure and camera pose from 2D images, using projections and multi-view geometric constraints.

Definitions

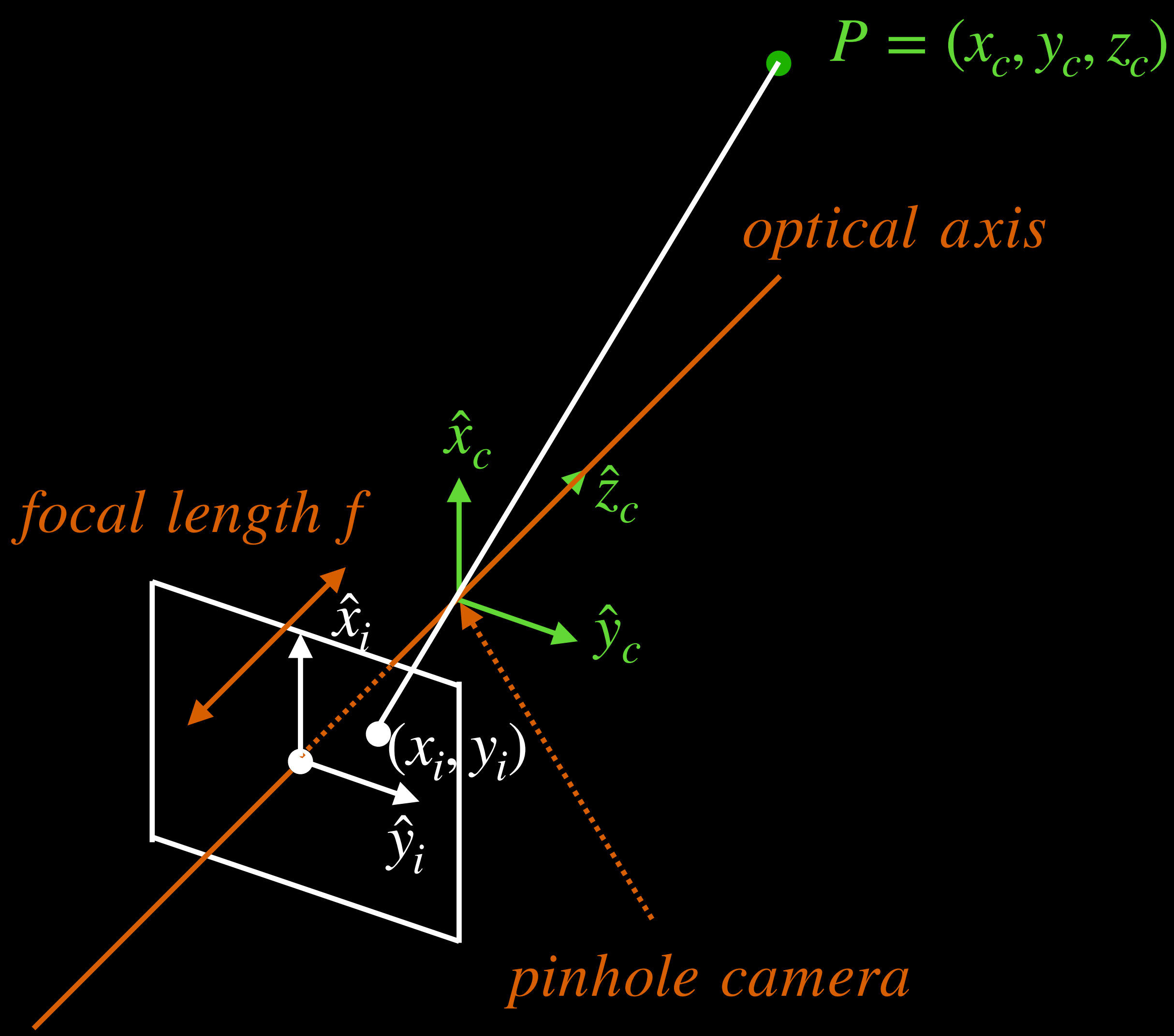
- Pose - 6 DoF rigid transform (R, t) describing camera position and orientation in the world coordinate system
- Projection - perspective mapping from 3D camera coordinates to 2D image coordinates
- Multi-view geometric constraint - epipolar geometry relating corresponding points across views from camera motion or the use of multiple cameras

The Process - 3D Point to 2D Image Sensor

Given a point P in space.

- 1) Map the 3D camera coordinate for point P to a point on the 2D image sensor using **perspective projection**.
- 2) Map the 3D world coordinate for point P to the 3D camera coordinate using **pose** (R, t).

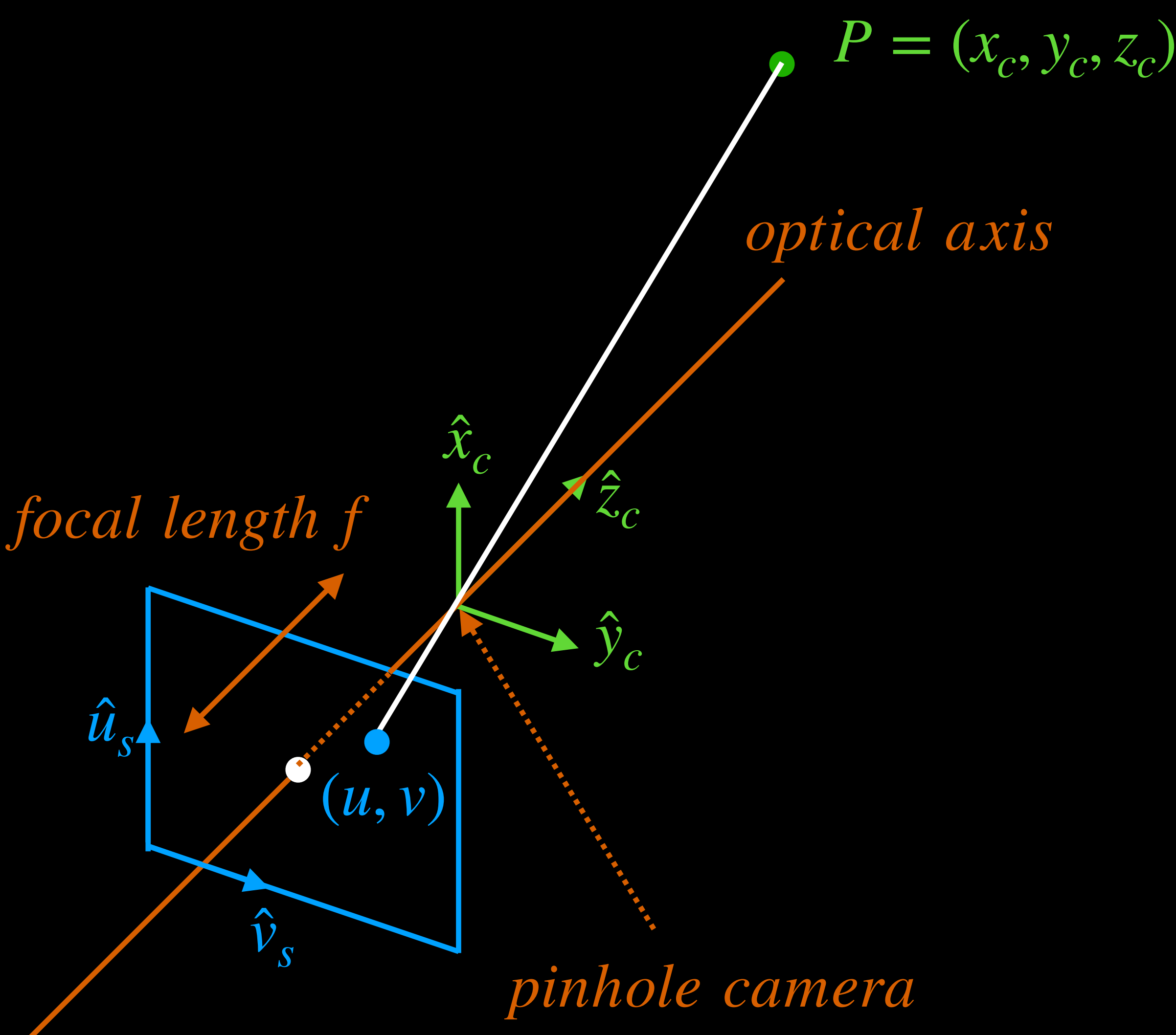
1) Camera Coordinate & Image Sensor (mm)



$$\frac{x_i}{f} = \frac{x_c}{z_c}, \quad \frac{y_i}{f} = \frac{y_c}{z_c}$$

$$x_i = f\frac{x_c}{z_c}, \quad y_i = f\frac{y_c}{z_c}$$

1) Camera Coordinate & Image Sensor (pixel)



$$x_i = f\frac{x_c}{z_c}, \quad y_i = f\frac{y_c}{z_c}$$

$$f_x = m_x f, \quad f_y = m_y f$$

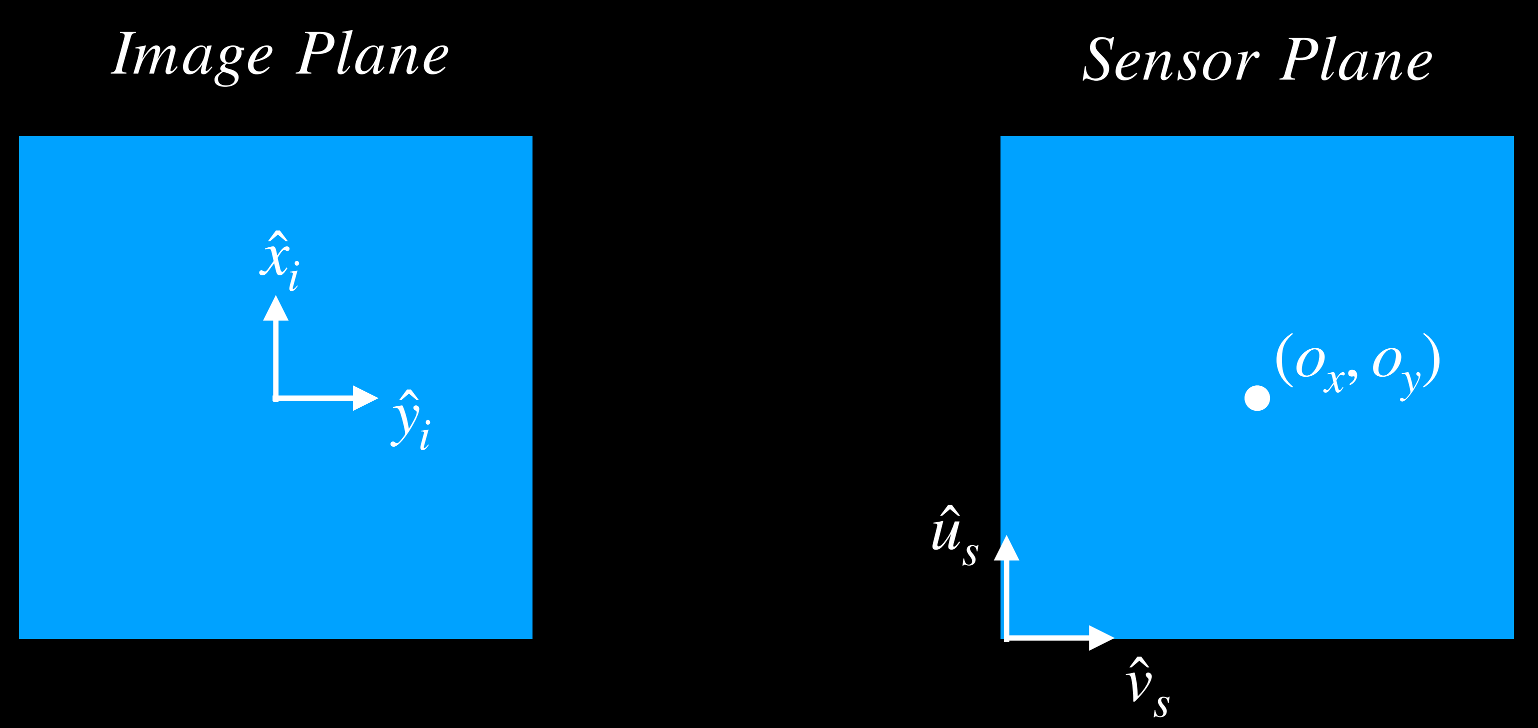
$$m_x, m_y \ units \rightarrow \frac{pixels}{mm}$$

Perspective Projection Equations

$$u = f_x\frac{x_c}{z_c} + o_x, \quad v = f_y\frac{y_c}{z_c} + o_y$$

$$optical\ center = (o_x, o_y)$$

Change of Coordinate System



The camera optical axis intersects the sensor plane at the optical center - (o_x, o_y)

Linearize the Perspective Projection Equations

How?

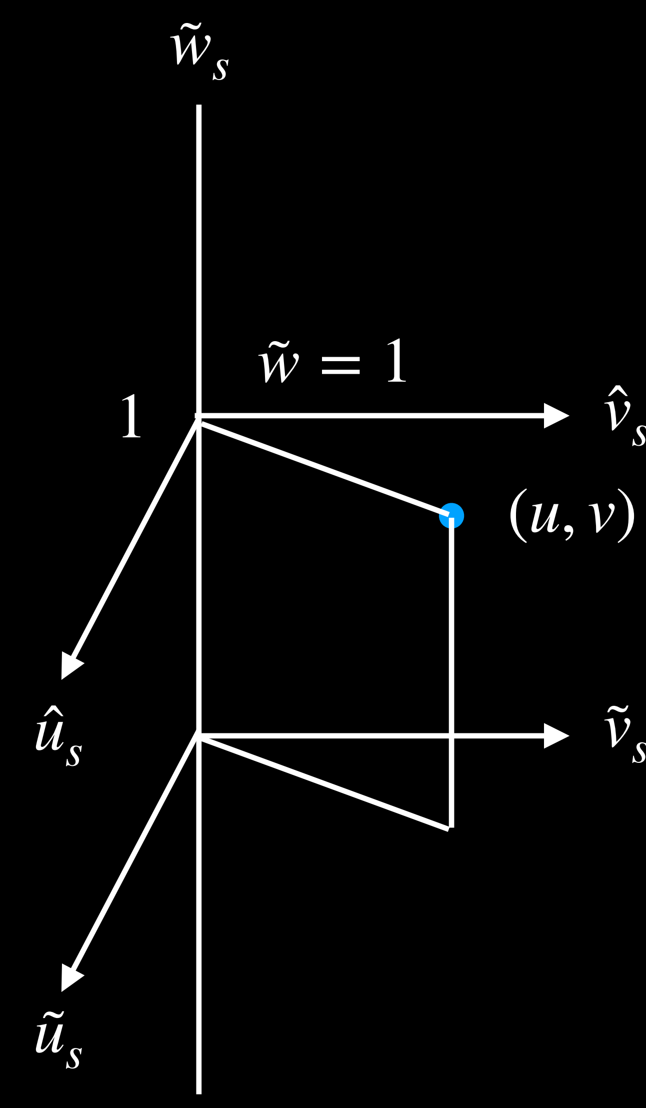
- Use Homogenous Coordinates.
- Homogeneous Coordinates introduces a scale factor $\tilde{w}$.
- With $\tilde{w}$, a 2D coordinate point becomes a 3D coordinate point.
- More importantly, we can linearize the perspective projection equations by letting $\tilde{w} = z_c$

Homogeneous Coordinates

$\hat{\mathbf{u}} = (u, v)$ *becomes* $\tilde{\mathbf{u}} = (\tilde{u}, \tilde{v}, \tilde{w})$

$$u = \frac{\tilde{u}}{\tilde{w}}, \quad v = \frac{\tilde{v}}{\tilde{w}}, \quad \tilde{w} \neq 0$$

$$\hat{\mathbf{u}} \equiv \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{w}u \\ \tilde{w}v \\ \tilde{w} \end{bmatrix} \equiv \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} \equiv \tilde{\mathbf{u}}$$



Homogenous coordinates adds an additional axis.

Putting It Together

$$u = f_x \frac{x_c}{z_c} + o_x, \quad v = f_y \frac{y_c}{z_c} + o_y$$

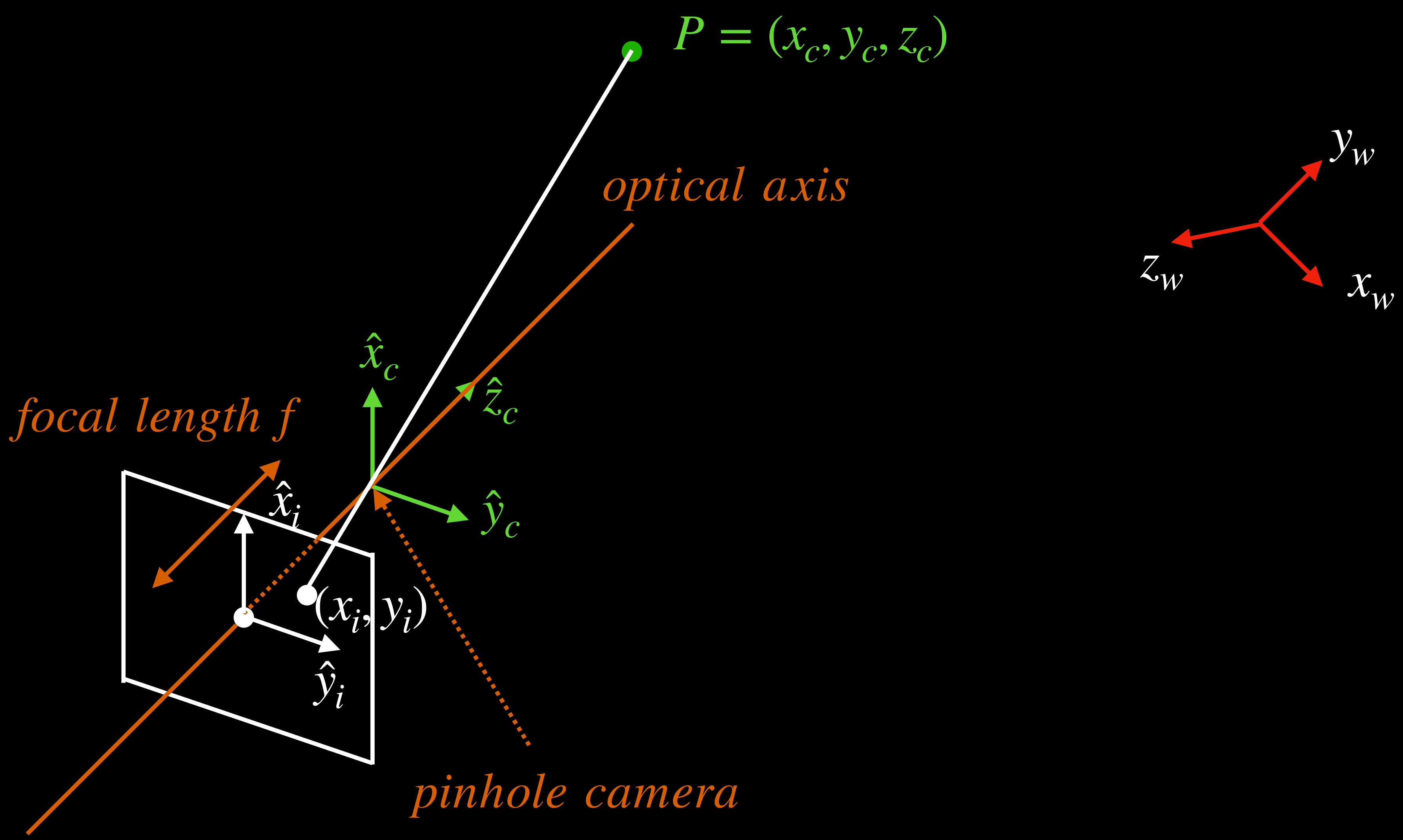
$$\hat{\mathbf{u}} \equiv \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ 1 \end{bmatrix} \equiv \underbrace{\begin{bmatrix} \tilde{w}u \\ \tilde{w}v \\ \tilde{w} \end{bmatrix} \equiv \begin{bmatrix} z_c u \\ z_c v \\ z_c \end{bmatrix}}_{Let\ \tilde{w} = z_c} \equiv \begin{bmatrix} f_x x_c + o_x \\ f_y x_c + o_y \\ z_c \end{bmatrix} \equiv \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

Using homogeneous coordinates, we can create a linear model that maps camera coordinates to sensor coordinates via perspective projection.

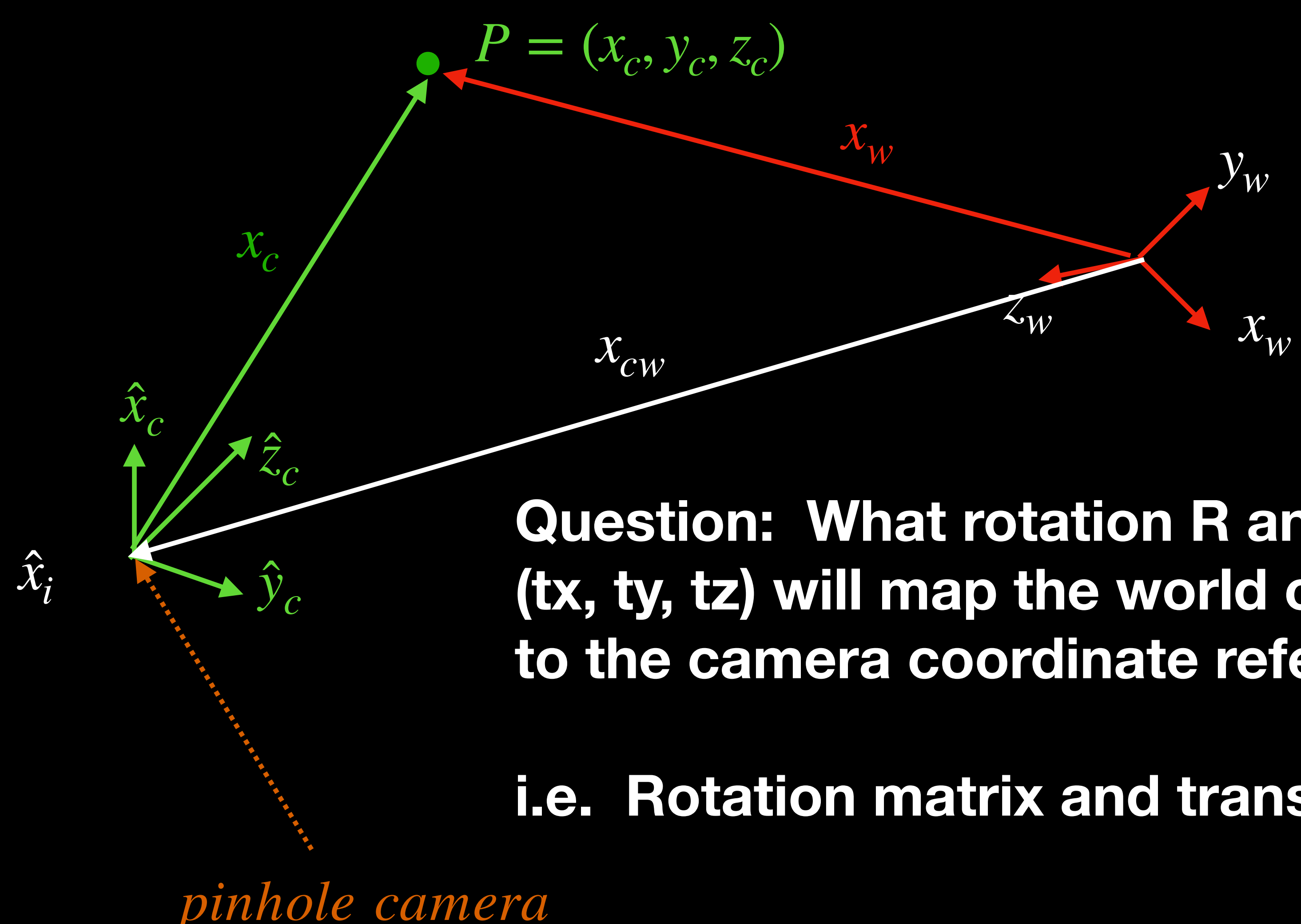
Definitions

Camera Intrinsic Matrix	**Camera Calibration Matrix**
$\begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$

2) World Coordinate to Camera Coordinate



2) World Coordinate to Camera Coordinate



Question: What rotation R and translation t = (tx, ty, tz) will map the world coordinate reference to the camera coordinate reference?

i.e. Rotation matrix and translation x_{cw} ?

2) World Coordinate to Camera Coordinate

$$Begin \rightarrow \ x_w$$

$$Subtract\ x_{cw} \rightarrow x_w - x_{cw}$$

$$Rotate\ the\ result \rightarrow R(x_w - x_{cw})$$

$$Yields \rightarrow Rx_w - Rx_{cw} = Rx_w + t, \quad t = -Rx_{cw}$$

$$\mathbf{x_c} = \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

In Homogeneous Coordinates, this can be rewritten as:

Camera Extrinsic Matrix

$$\tilde{\mathbf{x}}_{\mathbf{c}} = \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

The Mappings

Perspective Projection (Camera to Pixel)

$$\hat{\mathbf{u}} \equiv \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{w}u \\ \tilde{w}v \\ \tilde{w} \end{bmatrix} \equiv \begin{bmatrix} z_c u \\ z_c v \\ z_c \end{bmatrix} \equiv \begin{bmatrix} f_x x_c + o_x \\ f_y x_c + o_y \\ z_c \end{bmatrix} \equiv \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

Pose (World to Camera)

$$\tilde{\mathbf{x}}_\mathbf{c} = \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

End to End (World to Pixel)

$$\hat{\mathbf{u}} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \left(\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix} \right)$$

We have derived the end to end mapping. However, we do not know R and t.

Definition

Projection Matrix — **Intrinsic Matrix** — **Extrinsic Matrix**

$$P = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Projection Matrix unrolled to a column vector p

$$\mathbf{p} = [p_{11}, p_{12}, p_{13}, p_{14}, p_{21}, p_{22}, p_{23}, p_{24}, p_{31}, p_{32}, p_{33}, p_{34}\ ]^T$$

R and t are obtained by solving for the Projection Matrix P.

Solve for Projection Matrix

- Take a known 3D object with feature points and place it in the scene.
- Map the feature points in the 3D world coordinates to the 2D sensor coordinates.
- There are 12 unknown terms in the Projection Matrix P (->11 after normalization).
- Each feature point produces 2 equations.
- => Need at least 6 unique feature points (feature points cannot be coplanar or collinear) to solve for the Projection Matrix P.

Solve for Projection Matrix

- Write the equations for each feature point and aggregate the equation into a matrix multiply of the form A**p** = 0 (see appendix - its just math).

- **p** is the column vector associated with the unrolled Projection Matrix P.

- Apply the additional constraint: $\|\mathbf{p}\|^2 = 1$, since the solution to the equation above does not change if we use kP or P, for non-zero k.

- This results in the optimization problem:

$$\min_{\mathbf{p}} \|A\mathbf{p}\|^2 \ \textit{subject to} \ \|\mathbf{p}\|^2 = 1$$

$$\min_{\mathbf{p}}(\mathbf{p}^T A^T A\mathbf{p}) \ \textit{subject to} \ \mathbf{p}^T\mathbf{p} = 1$$

- Express this as a loss function:

$$L(p, \lambda) = \mathbf{p}^T A^T A\mathbf{p} - \lambda(\mathbf{p}^T\mathbf{p} - 1)$$

Solve for Projection Matrix

- Take the derivative of the loss function with respect to **p**:

$$\frac{dL(\mathbf{p}, \lambda)}{d\mathbf{p}} = 2A^T A\mathbf{p} - 2\lambda\mathbf{p} = 0$$

- This yields the classic eigenvalue problem: $A^T A\mathbf{p} = \lambda\mathbf{p}$

- We are interested in the smallest eigenvalue:

$$\mathbf{p}_{est} \leftarrow eigenvector\ associated\ with\ smallest\ eigenvalue\ \lambda\ of\ A^T A$$

- Now, re-roll the column vector $\mathbf{p}_{est}$ into the projection matrix P.

Compute the K and R Matrices from P

Projection Matrix — **Intrinsic Matrix** — **Extrinsic Matrix**

$$P = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

K — **R**

$$\begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

By applying RQ decomposition, we obtain K and R.
(Note: By construction, K is upper triangular and R is orthonormal.

What about the 3D translation t?

Translation

t can be "isolated" using the following matrix multiply:

$$\begin{array}{ccccc} \textbf{p4} & & \textbf{K} & & \textbf{t} \\ \begin{bmatrix} p_{14} \\ p_{24} \\ p_{34} \end{bmatrix} & = & \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} & & \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \end{array}$$

Then: $K^{-1}p4 = t$

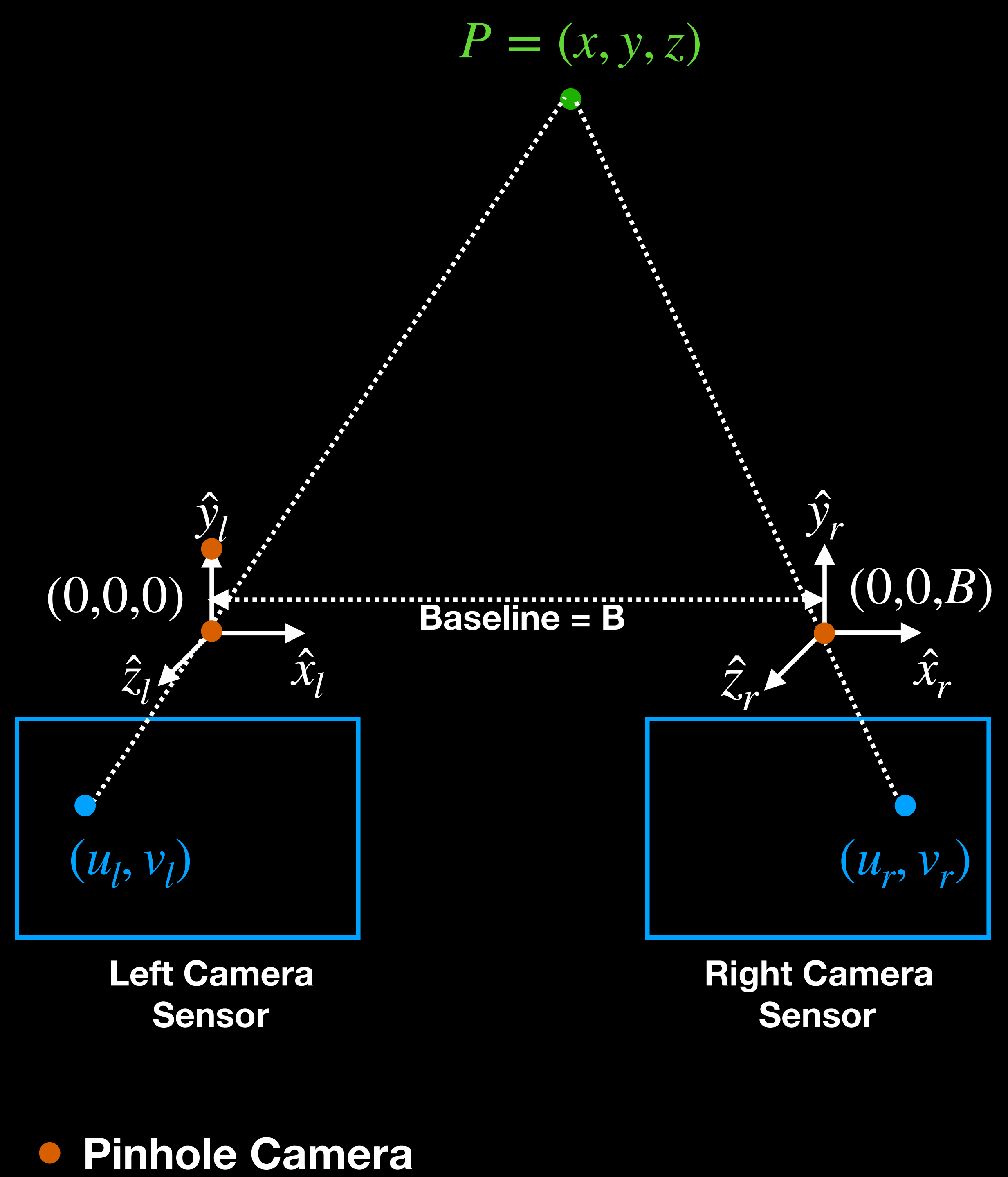
Multiview Geometry

- We have finished mapping the path from the world coordinate system to the sensor plane.
- In the process, we determined the appropriate rotation matrix **R** and translation vector **t**, mapping the world coordinate system to the camera coordinate system.
- Next, let's introduce a second camera.
- We begin with Multiview - Stereo.
- We then generalize to Multiview - Non Stereo.

Multiview - Stereo

- The simplest multiview geometry problem is stereo vision.
- Here, two identical cameras (same camera calibration matrices K) are placed side by side, separated by horizontal baseline.
- There is no vertical displacement between the two cameras.
- There is no rotation between the two cameras.
- The depth of a scene point (x,y,z) is computed from triangulation.

Multiview - Stereo



$$u_l = f_x \frac{x}{z} + o_x, \quad v_l = f_y \frac{y}{z} + o_y$$

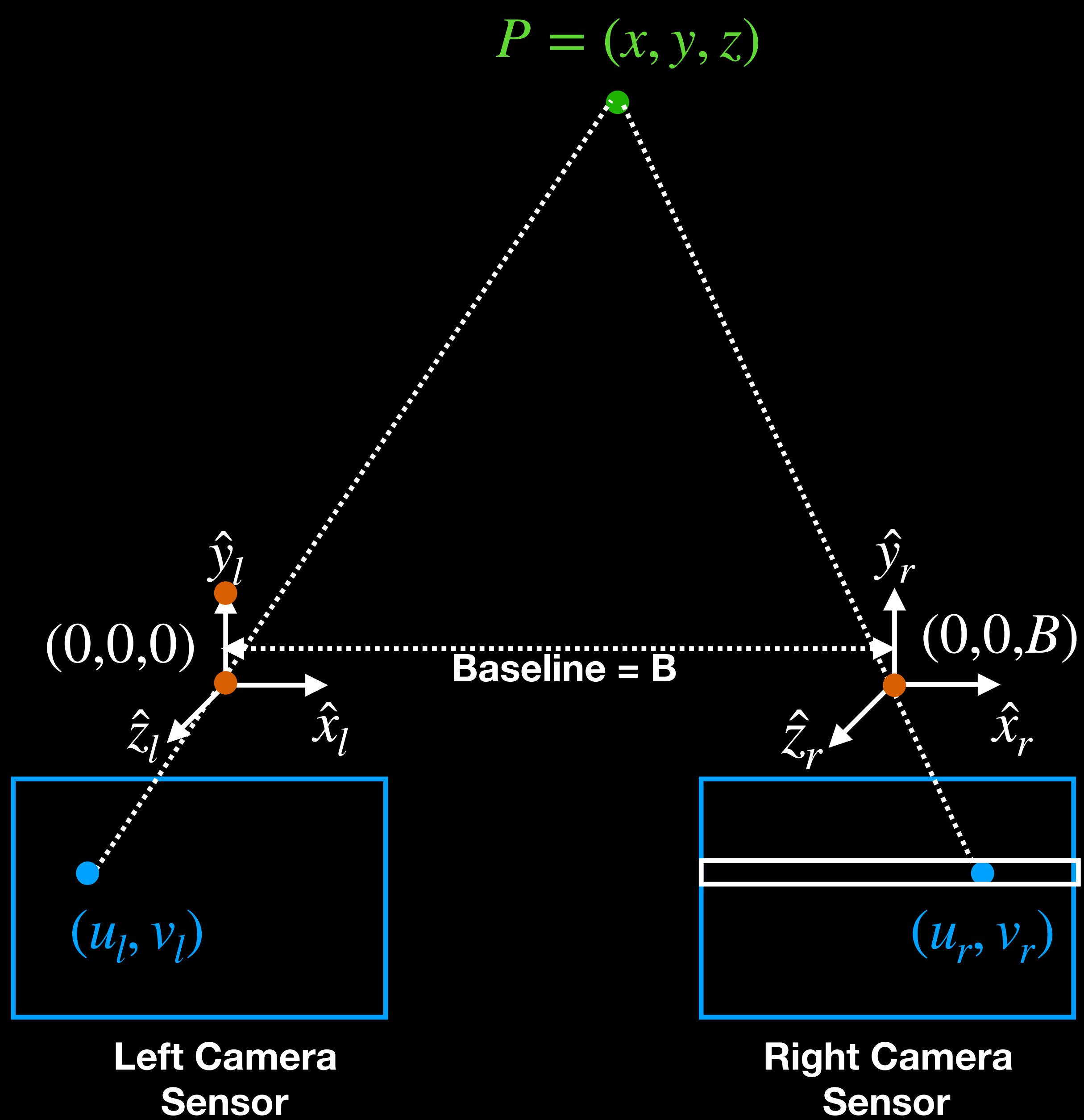
$$u_r = f_x \frac{(x - B)}{z} + o_x, \quad v_r = f_y \frac{y}{z} + o_y$$

$$\rightarrow x = \frac{B(u_l - o_x)}{(u_l - u_r)}$$

$$\rightarrow y = \frac{Bf_x(v_l - o_y)}{f_y(u_l - u_r)}$$

$$\rightarrow z = \frac{Bf_x}{(u_l - u_r)}$$

Multiview - Stereo



$$u_l = f_x \frac{x}{z} + o_x, \quad v_l = f_y \frac{y}{z} + o_y$$

$$u_r = f_x \frac{(x-B)}{z} + o_x, \quad v_r = f_y \frac{y}{z} + o_y$$

$$\rightarrow x = \frac{B(u_l - o_x)}{(u_l - u_r)}$$

$$\rightarrow y = \frac{Bf_x(v_l - o_y)}{f_y(u_l - u_r)}$$

$$\rightarrow z = \frac{Bf_x}{(u_l - u_r)}$$

● **Pinhole Camera**

Since the cameras have no vertical offset, the point u_r can be found by searching along a horizontal scanline in the right camera sensor.

In practice, this search is performed over a small window / patch.

Observations

- The depth z is an inverse function of: $Disparity = (u_l - u_r)$
- Close objects will have large disparity.
- Far objects will have small disparity.
- If the scene is at infinity, the images captured from the left and right cameras will be identical.

Observations

- The baseline B determines the accuracy of the depth.
- For a fixed z, a small B results in a small disparity.
- For fixed z, a large B results in a large disparity.
- For accurate z calculations, a large B is desirable.
- However 1) a large B may not be physically realizable and 2) a large B increases occlusion artifacts.

Multiview - Non Stereo

- Now, let’s consider the non stereo case.

- Here, we need to know the relationship of one camera to the other - i.e. the translation and rotation.

How?

- We will use of epipolar geometry, to constrain the problem.

- From the epipolar constraint, we will obtain the essential matrix E.

- E can be decomposed into a matrix containing the unknown translation terms and a matrix containing the rotation terms.

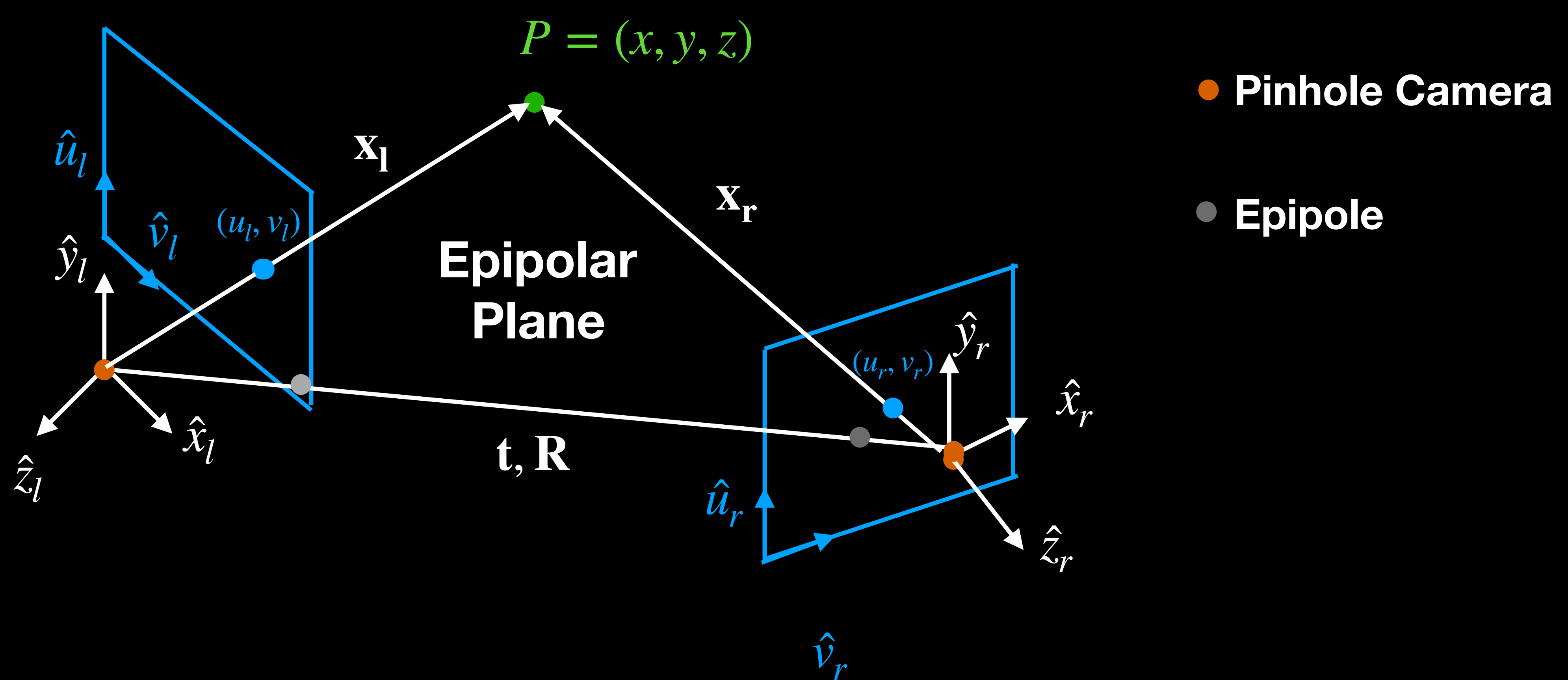
Multiview - Non Stereo

- Unfortunately, E will depend on 3D scene points.
- By performing an additional set of substitutions, we will map the 3D scene points to 2D sensor points.
- This results in a new relation involving sensor points, and a Fundamental Matrix F.
- Mathematically, F and E are related by the left and right camera intrinsic matrices.
- Since the camera intrinsic matrices are assumed known, E is obtained from the Fundamental Matrix F and the K’s.

Multiview - Non Stereo

- Singular Value Decomposition is applied to the Essential Matrix E.
- This results in 2 matrices - one containing translation information and one containing rotational information.

Epipolar Geometry

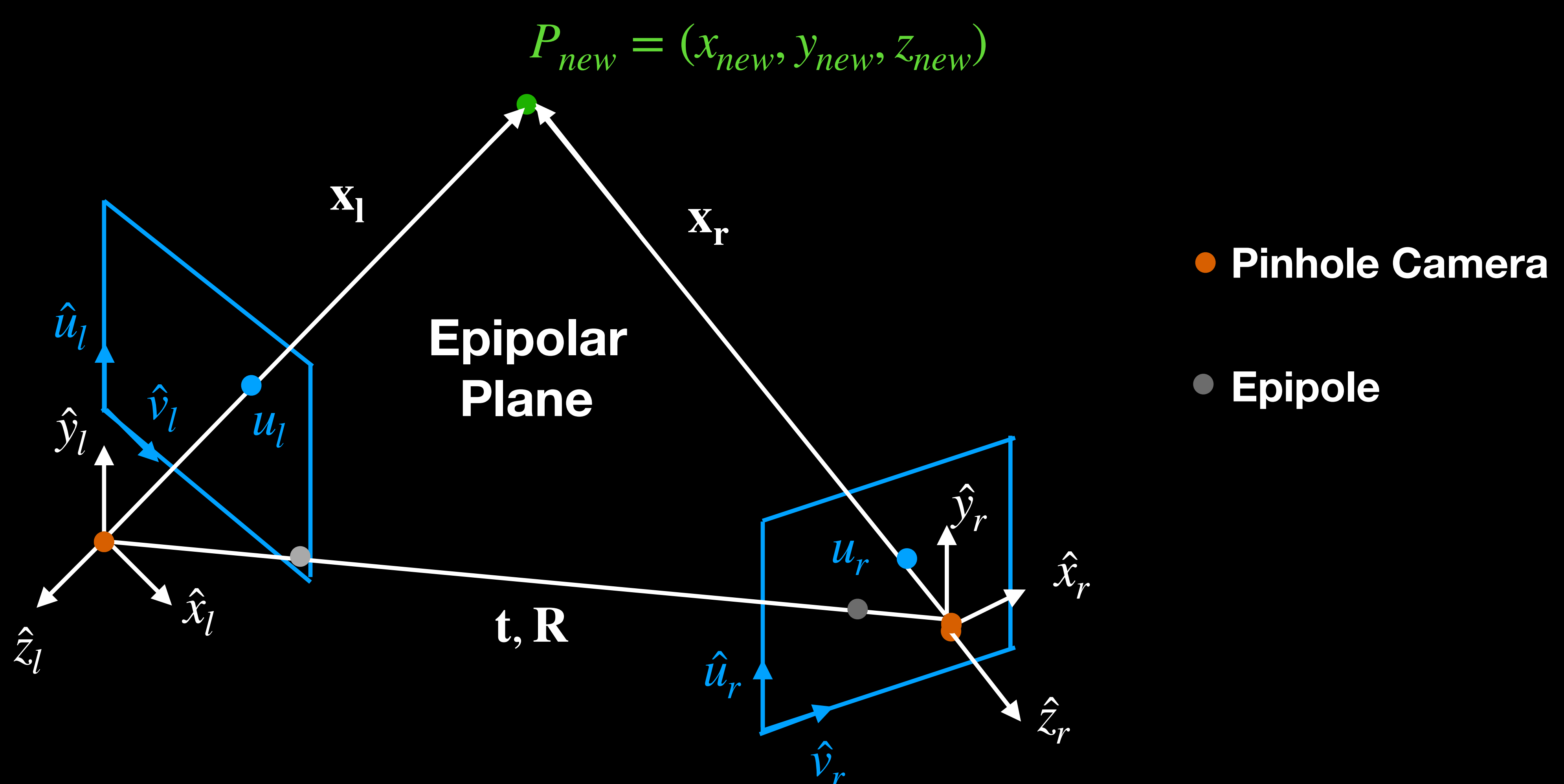


An epipolar plane is created from the pinhole cameras and the scene point.

The epipoles are the points where the baseline intersects the sensor planes.

A new epipolar plane is defined for every scene point, as shown on the next slide.

Epipolar Geometry



For the previous case of stereo vision, t=B and R=0.

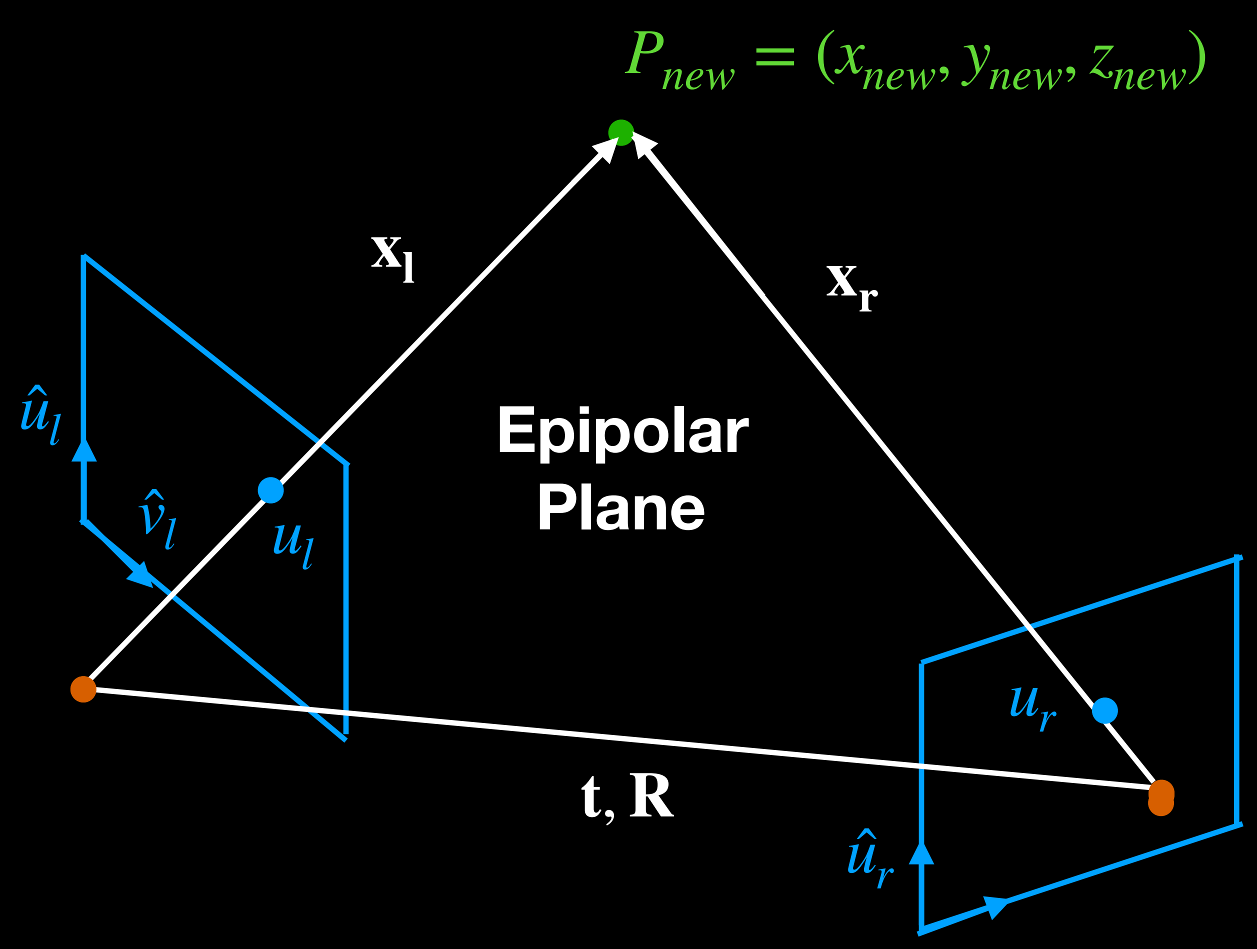
For the previous case of stereo vision, the epipoles were at infinity.

For the general multiview scenario, we need to determine t and R between the two cameras.

Essential Matrix

- The Essential Matrix E is derived from the epipolar constraint.
- The epipolar constraint is an “absolute truth” that results from epipolar geometry.
- In practice, we compute the Fundamental Matrix F, using the camera intrinsic matrices to obtain E.

Essential Matrix



$\mathbf{n} = \mathbf{t} \times \mathbf{x_l}$, *By construction* $\mathbf{n}$ *is perpendicular to the epipolar plane*

Epipolar Constraint

$\mathbf{x_l} \cdot (\mathbf{t} \times \mathbf{x_l}) = 0$

Essential Matrix

$$\mathbf{t} \times \mathbf{x_l} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ t_x & t_y & t_z \\ x_l & y_l & z_l \end{vmatrix} = (t_y z_l - t_z y_l)\hat{\mathbf{i}} - (t_x z_l - t_z x_l)\hat{\mathbf{j}} + (t_x y_l - t_y x_l)\hat{\mathbf{k}}$$

$$\mathbf{x_l} = (x_l,\ y_l,\ z_l), \quad \mathbf{x_l} \cdot (\mathbf{t} \times \mathbf{x_l}) = x_l(t_y z_l - t_z y_l) - y_l(t_x z_l - t_z x_l) + z_l(t_x y_l - t_y x_l)$$

$$\mathbf{x_l} \cdot (\mathbf{t} \times \mathbf{x_l}) = [x_l\ y_l\ z_l] \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \begin{bmatrix} x_l \\ y_l \\ z_l \end{bmatrix} = 0$$

$$\mathbf{x_l} = \mathbf{R}\mathbf{x_r} + \mathbf{t}, \quad \begin{bmatrix} x_l \\ y_l \\ z_l \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} x_r \\ y_t \\ z_r \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

Essential Matrix

$$\mathbf{x_l} \cdot (\mathbf{t} \times \mathbf{x_l}) = [x_l \; y_l \; z_l] \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \left(\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} x_r \\ y_t \\ z_r \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \right) = 0$$

$$\mathbf{x_l} \cdot (\mathbf{t} \times \mathbf{x_l}) = [x_l \; y_l \; z_l] \left(\overset{T_x}{\begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}} \overset{R}{\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}} \begin{bmatrix} x_r \\ y_t \\ z_r \end{bmatrix} + \underbrace{\cancel{\begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}}}_{\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}} \right) = 0$$

$$[x_l \; y_l \; z_l] \overset{E}{\begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix}} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} = 0$$

$$i.e. \; E = T_x R$$

Essential Matrix

$E = T_x R, \;\; T_x \; is \; skew \; symmetric : a_{ij} = -a_{ji}, \; R \; is \; orthonormal$

We can perform Singular Value Decomposition on E to obtain the two matrices.

However, we don’t know the 3D coordinate pairs.

Hence, we continue substituting, re-expressing the equation in sensor coordinates.

Recall

$$u_l = f_x^{(l)} \frac{x_l}{z_l} + o_x^{(l)}, \quad v_l = f_y^{(l)} \frac{y_l}{z_l} + o_y^{(l)}$$

$$z_l u_l = f_x^{(l)} x_l + z_l o_x^{(l)}, \quad z_l v_l = f_y^{(l)} y_l + z_l o_y^{(l)}$$

$$z_l \begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix} = \begin{bmatrix} z_l u_l \\ z_l v_l \\ z_l \end{bmatrix} \begin{bmatrix} f_x^{(l)} x_l + z_l o_x^{(l)} \\ f_y^{(l)} y_l + z_l o_y^{(l)} \\ z_l \end{bmatrix} = \begin{array}{c} K_l \\ \begin{bmatrix} f_x^{(l)} & 0 & o_x^{(l)} \\ 0 & f_y^{(l)} & o_y^{(l)} \\ 0 & 0 & 1 \end{bmatrix} \end{array} \begin{bmatrix} x_l \\ y_l \\ z_l \end{bmatrix}$$

Similarly:

$$z_r \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = \begin{bmatrix} z_r u_r \\ z_r v_r \\ z_r \end{bmatrix} \begin{bmatrix} f_x^{(r)} x_r + z_r o_x^{(r)} \\ f_y^{(r)} y_r + z_r o_y^{(r)} \\ z_r \end{bmatrix} = \begin{array}{c} K_r \\ \begin{bmatrix} f_x^{(r)} & 0 & o_x^{(r)} \\ 0 & f_y^{(r)} & o_y^{(r)} \\ 0 & 0 & 1 \end{bmatrix} \end{array} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix}$$

Recall

$$\begin{bmatrix} x_l \\ y_l \\ z_l \end{bmatrix} = K_l^{-1} z_l \begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix}, \quad \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} = K_r^{-1} z_r \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix}$$

$$\left(\begin{bmatrix} x_l \\ y_l \\ z_l \end{bmatrix} = K_l^{-1} z_l \begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix} \right)^T \rightarrow [x_l \;\; y_l \;\; z_l] = \begin{bmatrix} u_l & v_l & 1 \end{bmatrix} z_l \left(K_l^{-1} \right)^T$$

$$[x_l \;\; y_l \;\; z_l] \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} = 0$$

Substituting

$$\left(\begin{bmatrix} x_l \\ y_l \\ z_l \end{bmatrix} = K_l^{-1} z_l \begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix}\right)^T \rightarrow [x_l \ y_l \ z_l] = \begin{bmatrix} u_l & v_l & 1 \end{bmatrix} z_l \left(K_l^{-1}\right)^T$$

$$\begin{bmatrix} u_l & v_l & 1 \end{bmatrix} z_l \left(K_l^{-1}\right)^T \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} K_r^{-1} z_r \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0$$

$$\begin{bmatrix} u_l & v_l & 1 \end{bmatrix} \cancel{z_l} \left(K_l^{-1}\right)^T \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} K_r^{-1} \cancel{z_r} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0, \quad z_l \neq 0, \ z_r \neq 0$$

$$\begin{bmatrix} u_l & v_l & 1 \end{bmatrix} \underbrace{\left(K_l^{-1}\right)^T \begin{bmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{bmatrix} K_r^{-1}}_{Fundamental\ Matrix\ F} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0$$

$$\begin{bmatrix} u_l & v_l & 1 \end{bmatrix} \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & ef_{33} \end{bmatrix} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0$$

Recall

- Remember the machinery we used to solve for the Projection Matrix P? (~Slide 19 - Solve for Projection Matrix)
- We embark on a similar process, only now, for the Fundamental Matrix F.
- We take two images of the same scene captured from different viewpoints.
- We choose correspondence points, resulting in vector pairs.

Example

- $(u_l^{(1)}, v_l^{(1)}), \quad (u_r^{(1)}, v_r^{(1)})$
- $(u_l^{(2)}, v_l^{(2)}), \quad (u_r^{(2)}, v_r^{(2)})$
- $(u_l^{(3)}, v_l^{(3)}), \quad (u_r^{(3)}, v_r^{(3)})$

$\vdots$

- $(u_l^{(n)}, v_l^{(n)}), \quad (u_r^{(n)}, v_r^{(n)})$

For each correspondence point, we have a vector pair.

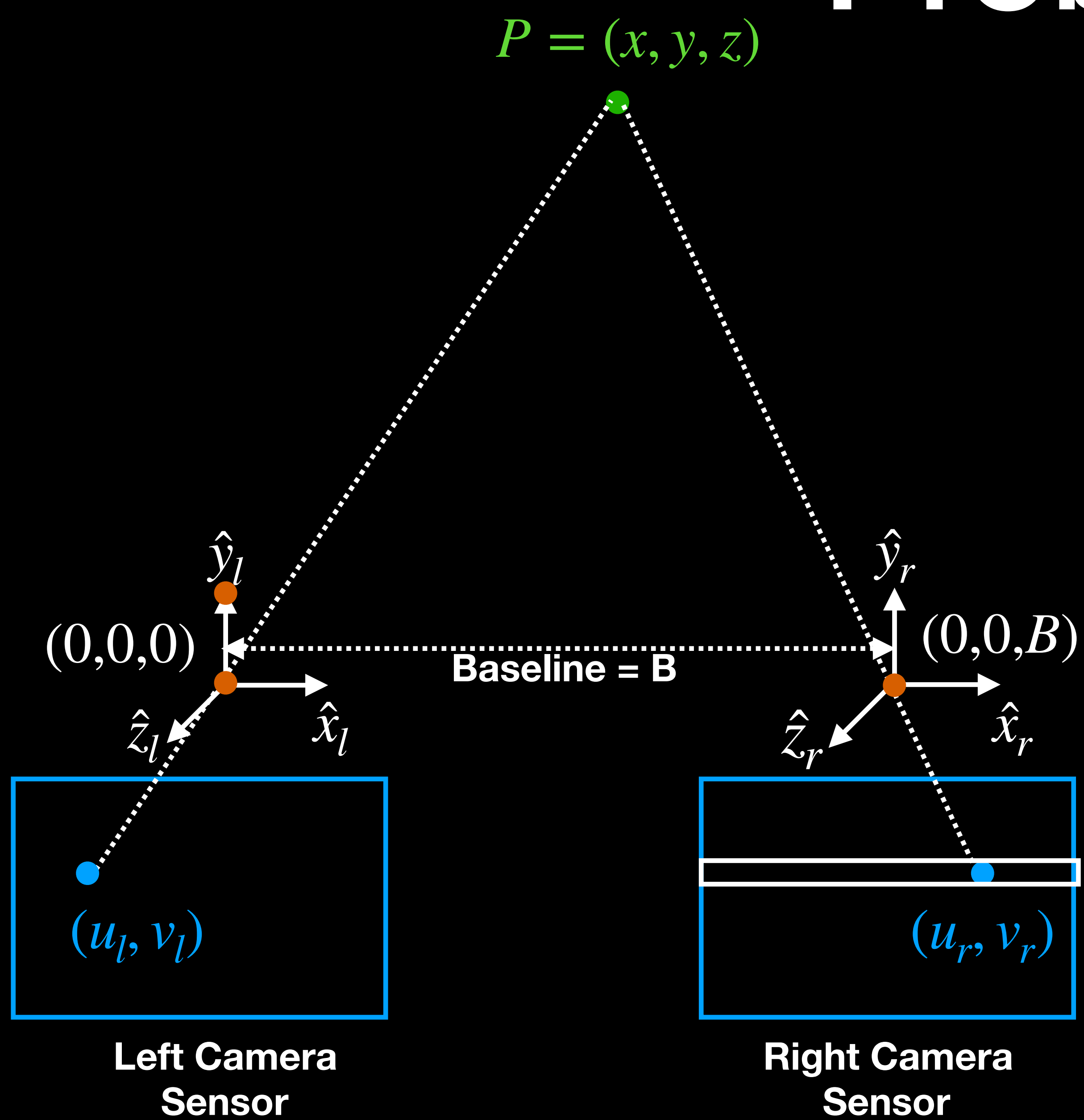
Example

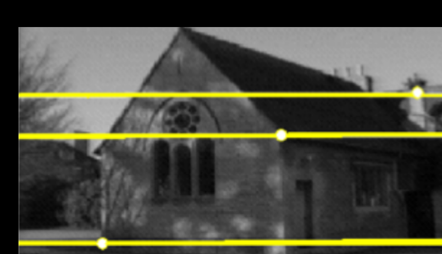
- We substitute the vector pairs into our previous equation with F, generating a system of equations.
- We rewrite the system of equations in matrix form as A**f** = 0 (see appendix).
- We rinse and repeat the process used to compute **p** for Projection Matrix P.
- Only now, we compute **f** for the Fundamental Matrix F.

Other Questions

- What other information can epipolar geometry reveal?
- What is the utility of the epipoles?

Recall: Stereo Correspondence Problem





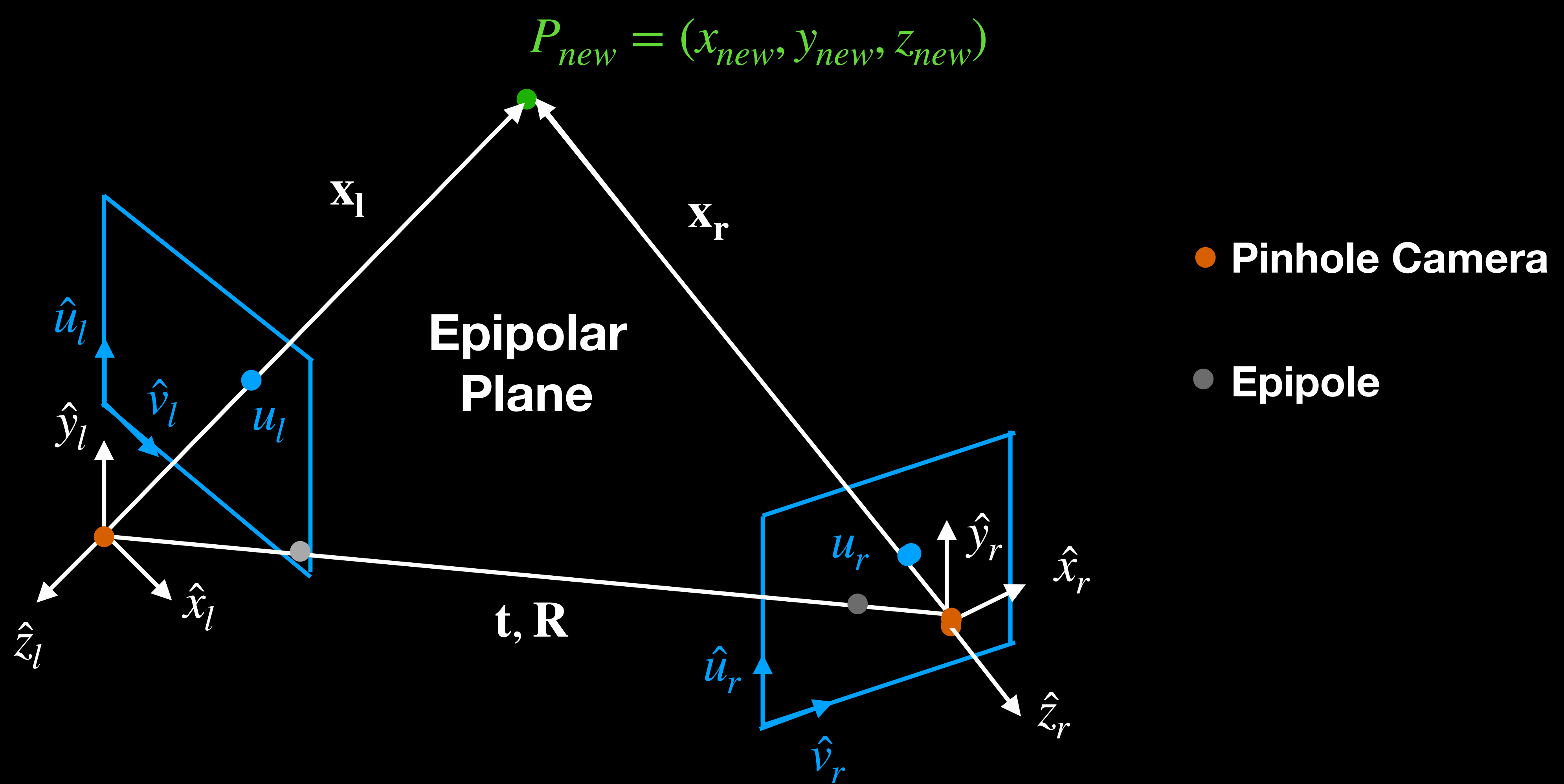
By constraining the problem, we were able to search for correspondences along horizontal scanlines.

How do we perform this search in the general case, when:

$$\mathbf{t} = (t_x, t_y \,.\, t_z) \neq (B,0,0)$$

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \neq \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Epipolar Geometry

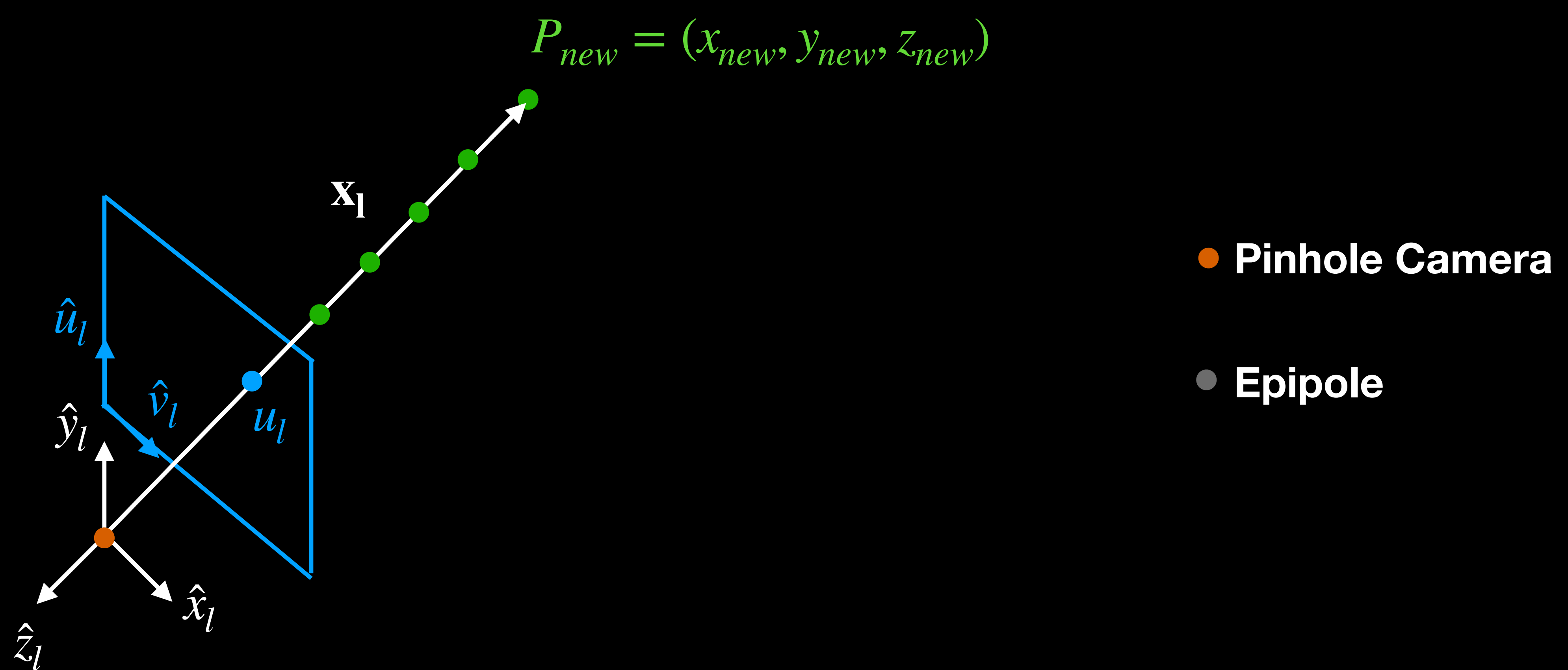


For stereo vision, t=B and R=0.

For stereo vision, the epipoles were at infinity.

For non stereo vision, the correspondence line for every point in the scene MUST pass through its the epipole.

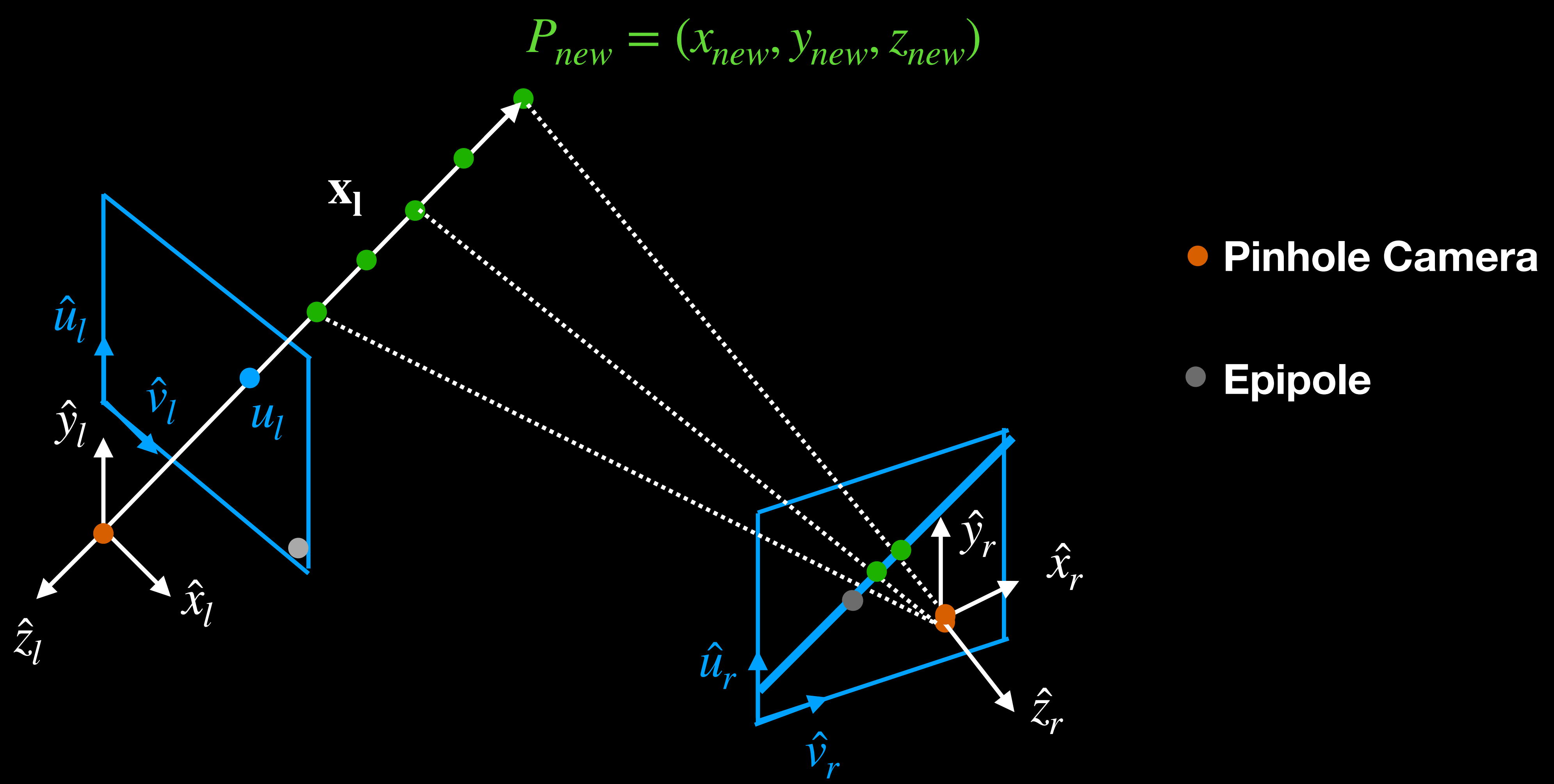
Epipolar Geometry



The imaged point u_l can lie at an infinite number of locations along the ray from from the left camera center to P_{new}.

When the points along this ray are projected through the pinhole aperture of the second camera, a "correspondence search line" is produced.

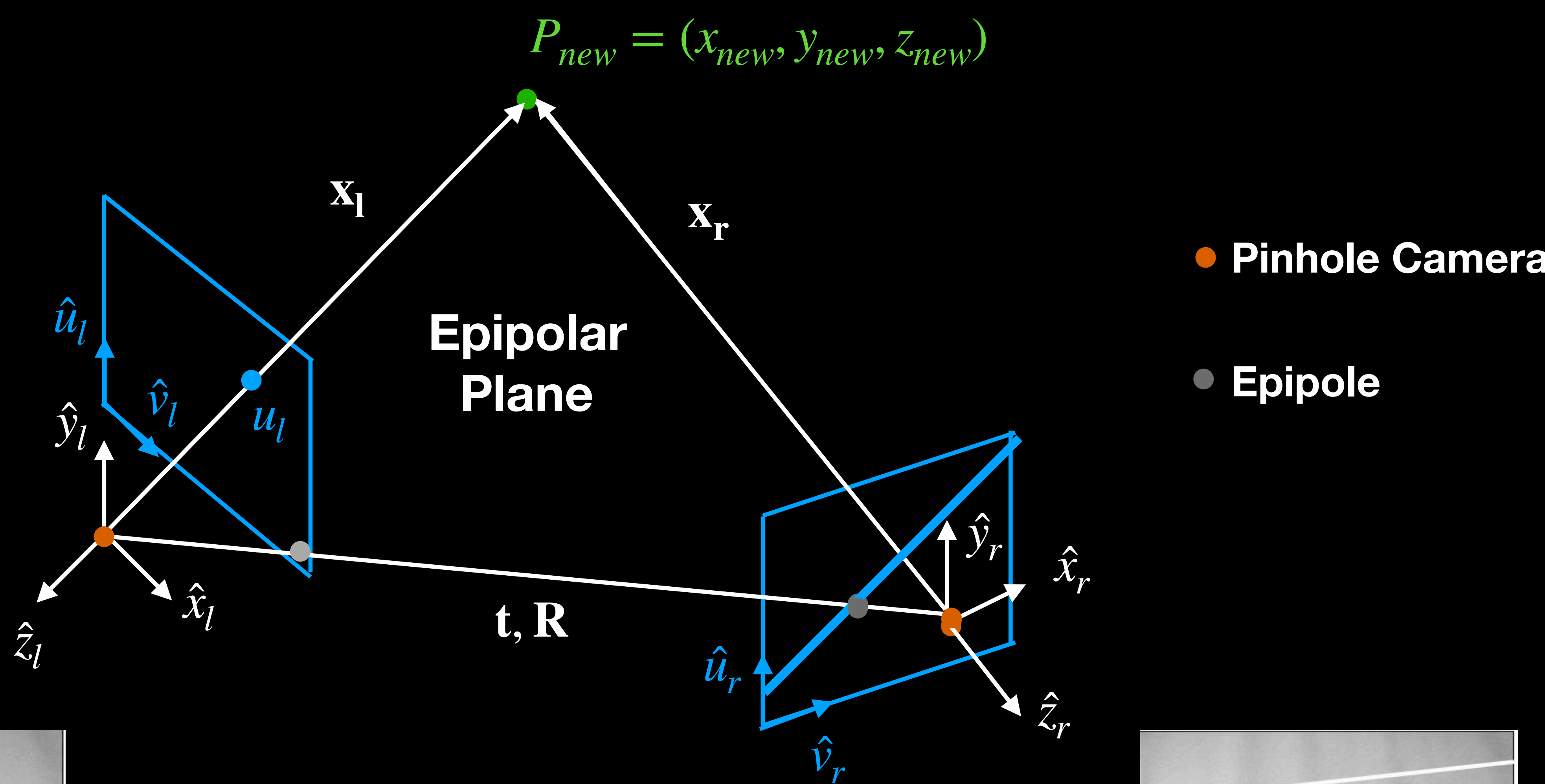
Epipolar Geometry

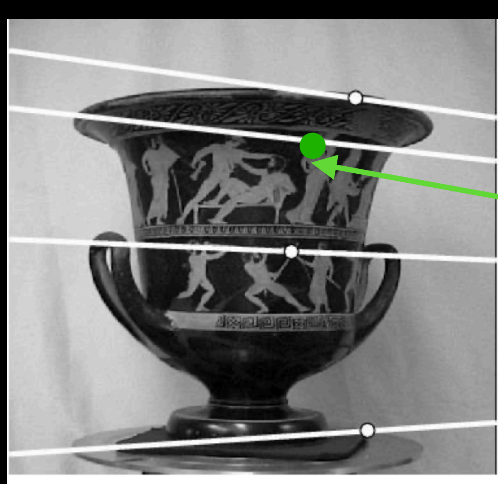


The correspondence line tells us where to search in the second camera sensor, to find the same scene point.

i.e. Search along the blue line.

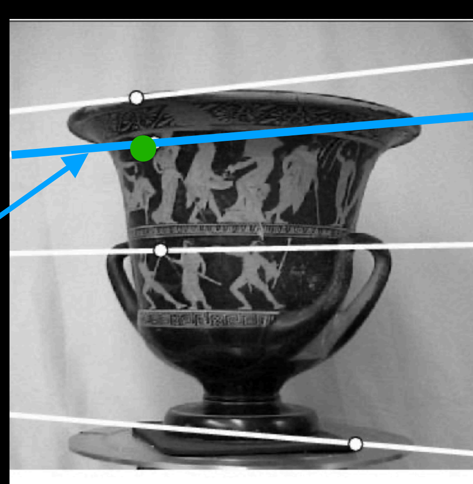
Non Stereo Correspondence Problem





Left image

Given this imaged point, search along the blue correspondence line for its match in the right image.



Right image

Equation for the Correspondence Line

$$\begin{bmatrix} u_l & v_l & 1 \end{bmatrix} \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & ef_{33} \end{bmatrix} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0$$

$$\begin{bmatrix} \overset{\textbf{a}}{(u_l f_{11} + v_l f_{21} + f_{31})} & \overset{\textbf{b}}{(u_l f_{12} + v_l f_{22} + f_{32})} & \overset{\textbf{c}}{(u_l f_{31} + v_l f_{32} + f_{33})} \end{bmatrix} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0$$

Equation for the correspondence search line in the right sensor for a given correspondence point in the left sensor. (F is known. a, b, c defined above.)

$$a u_r + b v_r + c = 0$$

Computing Scene Depth

$$\begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix} = \begin{bmatrix} f_x^{(l)} & 0 & o_x^{(l)} & 0 \\ 0 & f_y^{(l)} & o_y^{(l)} & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_l \\ y_l \\ z_l \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x_l \\ y_l \\ z_l \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix} = \begin{bmatrix} f_x^{(l)} & 0 & o_x^{(l)} & 0 \\ 0 & f_y^{(l)} & o_y^{(l)} & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \\ 1 \end{bmatrix}$$

$$\tilde{\mathbf{u}}_\mathbf{l} = P^{(l)} \tilde{\mathbf{x}}_\mathbf{r}$$

$$\begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = \begin{bmatrix} f_x^{(r)} & 0 & o_x^{(r)} & 0 \\ 0 & f_y^{(r)} & o_y^{(r)} & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \\ 1 \end{bmatrix}$$

$$\tilde{\mathbf{u}}_\mathbf{r} = M_{intrinsic}^{(r)} \tilde{\mathbf{x}}_\mathbf{r}$$

Computing Scene Depth

$$\tilde{\mathbf{u}}_{\mathbf{l}} = P^{(l)}\tilde{\mathbf{x}}_{\mathbf{r}}$$

$$\tilde{\mathbf{u}}_{\mathbf{r}} = M^{(r)}_{intrinsic}\tilde{\mathbf{x}}_{\mathbf{r}}$$

A **x** **=** **b**

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{43} \end{bmatrix} \begin{bmatrix} x_r \\ y_r \\ z_r \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}, \quad a_{ij} \; and \; b_i \; = f(M^{(r)}_{intrinsic}(m_{11 \; to \; 34}), P^{(l)}(p_{11 \; to \; 34}))$$

Compute pseudo inverse and solve for x

$$A^T A x = A^T b$$

$$x = (A^T A)^{-1} A^T b$$

Summary

- Using geometry, we constructed the perspective projection equations, mapping a 3D point in camera coordinates to a 2D point in sensor coordinates.
- Assuming rigid body motion, we mapped our camera coordinate system to our world coordinate system via (R,t).
- Casting everything in terms of homogeneous coordinates, we then derived a linear end-to-end mapping from world coordinates to sensor coordinates.

Summary

- We then examined the two multiview problems: stereo and non stereo.
- We observed how correspondence matching simplified to a simple horizontal scanline search in stereo multiview.
- For non-stereo multiview, we exploited epipolar geometry and the epipolar constraint.
- We computed the Essential Matrix E and the Fundamental Matrix F.

Summary

- From E and F, we were able to determine the equation of the correspondence line in the right (left) image sensor for a correspondence point in the left (right) image sensor.

- Given 1) matching 2D image sensor coordinates points and 2) using all of the machinery previously learned, we then reconstructed the corresponding 3D scene point, imaged on the two sensors.

Next Steps

- Kalman filters are recursive state estimators (see: /vision/slides/VISION_Kalman_Filter or /reinforcementLearning/slides/MODERN_CONTROL_Kalman_Filter)
- For computer vision, Kalman filters provide real time estimates of camera pose and velocity.
- Because Kalman filters operate incrementally and do not revisit past estimates, they can accumulate drift.
- Bundle adjustment performs nonlinear optimization over multiple frames to jointly revise camera pose and 3D scene points, enforcing geometric consistency.
- Bundle adjustment is more computationally expensive than Kalman filtering.
- The refined estimates from bundle adjustment are typically used (when available) to reduce drift in the Kalman filter.